

Protocol CDAP/1.0

Networks - Project 1: Socket Programming

จัดทำโดย

นายพลัฏฐ์ เจตน์มคลรัตน์ 6710405419

เสนอ

ผศ.ดร. สุขุมาล กิตติสิน

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

Computer Communications and Cloud Computing

Principles

ภาคการเรียนที่ 1

ปี 2569

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อเป็นคู่มือประกอบการศึกษาและการใช้งาน CDAP Protocol Version 1.0 (CDAP/1.0) โดยรวบรวมรายละเอียด หลักการทำงาน โครงสร้าง และแนวทางการใช้งานของโปรโตคอล เพื่อให้ผู้อ่านสามารถทำความเข้าใจลำดับกระบวนการสื่อสาร และวิธีการทำงานของระบบได้อย่างเป็นขั้นตอน

เนื้อหาภายในรายงานครอบคลุมตั้งแต่แนวคิดและองค์ประกอบพื้นฐานของ CDAP/1.0 รูปแบบการรับส่งข้อมูล กระบวนการทำงาน ตลอดจนวิธีการนำโปรโตคอลไปใช้งานในสถานการณ์ต่าง ๆ โดยมีจุดมุ่งหมายเพื่อให้รายงานฉบับนี้สามารถใช้เป็นทั้งเอกสารอ้างอิงและคู่มือสำหรับการศึกษา ทดลอง และพัฒนาระบบที่เกี่ยวข้องกับ CDAP/1.0 ได้อย่างถูกต้องและมีประสิทธิภาพ

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านมีความเข้าใจเกี่ยวกับ CDAP/1.0 มากยิ่งขึ้น และสามารถนำความรู้ที่ได้รับไปประยุกต์ใช้ในการศึกษาและพัฒนาระบบต่อไป

1. วัตถุประสงค์

CDAP ถูกพัฒนาขึ้นเพื่อเป็นแพลตฟอร์มการแข่งขันเขียนโปรแกรมที่มุ่งเน้นการพัฒนาทักษะด้านการแก้ไขปัญหาและการออกแบบอัลกอริทึมอย่างมีประสิทธิภาพ โดยพิจารณาผลลัพธ์จากทั้ง Time Complexity และ Space Complexity เพื่อส่งเสริมให้ผู้ใช้งานตระหนักถึงประสิทธิภาพของโปรแกรม นอกเหนือจากความถูกต้องของผลลัพธ์เพียงอย่างเดียว

ระบบรองรับทั้ง โหมดผู้เล่นเดี่ยว (Single-player) สำหรับฝึกฝนและพัฒนาทักษะด้วยตนเอง และ โหมดผู้เล่นสองคน (Two-player) สำหรับการแข่งขันภายใต้ข้อจำกัดด้านเวลา เพื่อจำลองสถานการณ์ที่ต้องอาศัยการคิด วิเคราะห์ และแก้ไขปัญหาอย่างรวดเร็ว โดยมีแนวคิดในการนำองค์ประกอบของเกมมาใช้เพื่อสร้างความสนุก ความท้าทาย และแรงจูงใจในการพัฒนาทักษะการเขียนโปรแกรม

2. สถาปัตยกรรม

CDAP มีลักษณะเป็น Real-time, Interactive, Multi-client, Stateful, Reliable for critical messages และ Low-latency for live updates

ระบบ CDAP/1.0 ใช้สถาปัตยกรรมแบบ Client-Server โดยมี Arena Server เป็นศูนย์กลางในการควบคุมและประสานงานระหว่างผู้เล่นระบบคิว และ Judge Worker แต่ละเครื่อง

Player A --TCP--> <--TCP-- Judge Worker 1

Player B --TCP--> Arena Server + Queue <--TCP-- Judge Worker 2

Feed Pane <--UDP-- <--TCP-- Judge Worker N

Arena Server ทำหน้าที่เป็นหน่วยงานหลัก (Authority) ของระบบ โดยรับผิดชอบการจัดการข้อมูลและสถานะที่เกี่ยวข้องกับ Account, Lobby, Room, Match, Submission และ Verdict รวมถึงการจัดคิวงานสำหรับการตรวจสอบคำตอบของผู้เล่น

ในส่วนของ Judge Worker จะทำงานในลักษณะ Client โดยเป็นฝ่ายสร้างการเชื่อมต่อ TCP เข้ามายัง Arena Server และร้องขอชุดงานผ่านคำสั่ง WORKER_PULL ในรูปแบบ Long Polling เมื่อมีงานสำหรับตรวจสอบ Arena Server จะส่งรายละเอียดของ Submission ให้ Worker ประมวลผล และ Worker จะส่งผลการตัดสินกลับมายัง Arena Server

แนวทางดังกล่าวทำให้ Judge Worker สามารถทำงานอยู่ภายใต้เครือข่ายที่มี NAT หรือ Firewall ได้ โดยไม่จำเป็นต้องเปิด Inbound Port สำหรับการเชื่อมต่อจากภายนอก เนื่องจากการเชื่อมต่อทั้งหมดเริ่มต้นจากฝั่ง Worker ไปยัง Arena Server

สำหรับ Feed Pane ระบบใช้ UDP สำหรับรับข้อมูลอัปเดตที่ไม่จำเป็นต้องรับประกันการส่งถึงทุกแพ็กเก็ต เช่น ข้อมูลสถานะการแข่งขัน หรือข้อมูลที่ใช้สำหรับการแสดงผลแบบ Real-time ซึ่งช่วยลดภาระในการสร้างและรักษาการเชื่อมต่อ

3. คำอธิบายคำสั่ง

Command	คำอธิบาย
help	แสดงรายการคำสั่งที่สามารถใช้ได้
queue	เข้าสู่คิวการหาห้องออนไลน์
dequeue	ออกจากคิวก่อนเริ่มเกม
room	สร้างห้องส่วนตัว
room fib	สร้างห้องส่วนตัว พร้อมระบุโจทย์ที่ต้องการใช้ในการแข่งขัน
join ABCD	เข้าห้องส่วนตัวด้วย code

ready	กำหนดสถานะของตนเองว่า พร้อมเล่น
leave	ออกจากห้องปัจจุบัน
problem	แสดงรายละเอียดที่พร้อมใช้ในการแข่งขัน
submit solution.py	ส่งโค้ดคำตอบเข้าสู่กระบวนการตรวจคำตอบ
status	ตรวจสอบสถานะการส่งคำตอบล่าสุด
status s-0001	ตรวจสอบสถานะการส่งคำตอบตาม Session Id ที่ระบุ
forfeit	ยอมแพ้และสิ้นสุดการแข่งขัน
whoami	แสดงข้อมูลผู้ใช้, สถานะการแข่งขัน, ห้องปัจจุบัน, สถานะการส่งคำตอบ

quit or exit	ออกจากระบบและยุติการเชื่อมต่อ
panic	คำสั่งสำหรับการทดสอบ และบังคับให้ server ส่งกลับมา
raw METHOD {"key": "value"}	ส่ง request ไปยัง server โดยตรง ใช้สำหรับการทดสอบโดยตรง

4. คำสั่งเริ่มการทำงาน

คำสั่ง	คำอธิบาย
<pre>py -3.14 -m cdap.client --user alice --pass 1234 --queue</pre>	เข้าสู่ระบบและเข้าคิวค้นหาคู่แข่งทันที
<pre>py -3.14 -m cdap.client --user alice --pass 1234 --queue --submit solution.py --once</pre>	เข้าคิวและส่งไฟล์คำตอบโดยอัตโนมัติเมื่อการแข่งขันเริ่มต้น
<pre>py -3.14 -m cdap.client --user alice --pass 1234 --no-udp</pre>	ปิดการรับข้อมูลอัปเดตแบบ Live ผ่าน UDP
<pre>py -3.14 -m cdap.client --user alice --feed-only</pre>	เปิดหน้าจอสำหรับแสดงคะแนนและเวลาการแข่งขันผ่าน UDP เท่านั้น

<pre>py -3.14 -m cdap.client --user alice --pass 1234 --wire</pre>	แสดงรายละเอียด Protocol Frame ที่รับส่งผ่าน TCP และ UDP ทั้งหมด
<pre>py -3.14 -m cdap.server</pre>	เริ่มต้น Server สำหรับการแข่งขันแบบผู้เล่น 2 คน
<pre>py -3.14 -m cdap.server --min-players 1</pre>	เริ่มต้น Server สำหรับการแข่งขันแบบผู้เล่นคนเดียว
<pre>py -3.14 -m cdap.server --min-players 1 --problem fib</pre>	กำหนดให้ทุกการแข่งขันใช้โจทย์ fib
<pre>py -3.14 -m cdap.server --min-players 1 --match-seconds 60 --countdown 3</pre>	กำหนดระยะเวลาการแข่งขันเป็น 60 วินาทีและกำหนดเวลานับ ถอยหลังก่อนเริ่มการแข่งขันเป็น 3 วินาที
<pre>py -3.14 -m cdap.server --backend docker</pre>	ใช้ Docker เป็น Backend สำหรับแยกสภาพแวดล้อมในการประมวล ผลโค้ดที่ผู้เล่นส่งเข้ามา

<code>py -3.14 -m cdap.server --help</code>	แสดงตัวเลือกและพารามิเตอร์ทั้งหมดของ Server
<code>py -3.14 -m cdap.client --help</code>	แสดงตัวเลือกและพารามิเตอร์ทั้งหมดของ Client

5. การเลือก Transport Layer Service

Traffic	Transport Protocol	เหตุผล
HELLO, Auth, Match Making, Judge Verdict, Match Events, Worker Jobs	TCP	ข้อมูลมีความสำคัญและต้องถูกส่งถึงปลายทาง อย่างครบถ้วน รวมถึงต้องรักษาลำดับของ ข้อมูลให้ถูกต้อง
TICK, CLOCK, BOARD, Live Display	UDP	อัปเดตข้อมูลอย่างต่อเนื่องและให้ความสำคัญ กับความเร็ว เนื่องจากข้อมูลสามารถใช้ แทนข้อมูลเดิมได้

6. Protocol Status

Code	Status
200	OK
201	CREATED, REGISTERED
202	ACCEPTED, QUEUED
204	NO_CONTENT
400	BAD_REQUEST
401	AUTH_FAILED

403	FORBIDDEN, NOT_IN_MATCH, NOT_IN_ROOM, WRONG_STATE
408	REQUEST_TIMEOUT
409	CONFLICT, ALREADY_QUEUED, NOT_QUEUED, ROOM_FULL, USER_EXISTS
410	MATCH_ENDED
413	PAYLOAD_TOO_LARGE
415	UNSUPPORTED_LANGUAGE
422	BODY_HASH_MISMATCH
426	VERSION_UNSUPPORTED

429	RATE_LIMITED, SUBMIT_COOLDOWN
500	INTERNAL_ERROR
503	JUDGE_UNAVAILABLE

7. Judge Verdict

Code	Phrase	ความหมาย
600	ACCEPTED	ถูกต้องและผ่าน complexity contract
601	WRONG_ANSWER	output value ผิด
602	TIME_LIMIT_EXCEEDED	parent kill เมื่อเกิน wall-clock
603	MEMORY_LIMIT_EXCEEDED	memory เกิน absolute cap
604	COMPILE_ERROR	parse/compile ไม่สำเร็จ
605	RUNTIME_ERROR	exception ระหว่างทำงาน

606	TIME_COMPLEXITY_VIOLATION	ถูกแต่ growth ช้ากว่า contract
607	SPACE_COMPLEXITY_VIOLATION	space growth เกิน contract
608	OUTPUT_FORMAT_ERROR	value เท่ากันแต่ type/shape ผิด
609	SANDBOX_VIOLATION	AST guard ปฏิเสธ operation
611	INDETERMINATE_COMPLEXITY	ไม่มี model ที่ fit อย่างน่าเชื่อถือ
612	JUDGE_ERROR	Judge ล้มเหลว ไม่ใช่ความผิดผู้เล่น

8.Complexity Measurement

8.1 Method A - Wall - Clock Regression

รันทก input sizes, warm-up หนึ่งครั้ง, วัดซ้ำห้าครั้งและใช้ค่าต่ำสุดเพื่อลด one-sided timing noise จาก scheduling ใช้ least squares through origin และ relative RMSE เปรียบเทียบ $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$

8.2 Method B - Opcode Counting

นับ Python bytecode instruction ด้วย sys.monitoring (PEP 669) และใช้หนึ่ง run ต่อ size เพราะ count deterministic Capability probe พบว่า sys.settrace/f_trace_opcodes บน CPython 3.14.3 เครื่องนี้คืน count เป็น ศูนย์แบบ silent จึงห้ามใช้ mechanism ที่ไม่ผ่าน non-zero, reproducible และ scaling checks

8.3 Decision Policy

- หาก Margin ≥ 1.15 ให้ถือว่าผลการจำแนกมีความเชื่อมั่นสูง (High Confidence)
- หาก Margin < 1.15 ให้เลือก Complexity Class ที่มีต้นทุนต่ำกว่า และระบุผลว่าเป็น Low Confidence
- หากค่า Best Relative RMSE > 0.35 ให้คืนผลลัพธ์ 611 INDETERMINATE_COMPLEXITY เนื่องจากข้อมูลไม่สอดคล้องกับ Complexity Class ที่ระบบรองรับอย่างเพียงพอ

9. Sandbox Backends

ระบบรองรับ Sandbox Backend สำหรับประมวลผลโค้ดที่ผู้เล่นส่งเข้ามา 2 รูปแบบ ได้แก่ SubprocessBackend และ DockerBackend โดยมีวัตถุประสงค์เพื่อแยกการทำงานของโค้ดผู้เล่นออกจากกระบวนการหลักของ Judge และลดผลกระทบที่อาจเกิดจากโค้ดที่ทำงานผิดพลาดหรือไม่พึงประสงค์

9.1 SubprocessBackend

SubprocessBackend จะสร้าง Process ใหม่สำหรับการประมวลผล Submission แต่ละครั้ง พร้อมสร้าง Temporary Working Directory แยกต่างหากสำหรับแต่ละงาน โดย Runner จะอ่านข้อมูลจาก stdin ให้เสร็จสิ้นก่อนโหลดและเรียกใช้งานโค้ดของผู้เล่น

ระบบกำหนดขนาดข้อมูล Output สูงสุดไว้ที่ 64 KB เพื่อป้องกันการสร้าง Output ปริมาณมากเกินไปจนความจำเต็ม และใช้กลไก Wall-clock Timeout ร่วมกับการยุติ Process Tree เมื่อโปรแกรมทำงานเกินระยะเวลาที่กำหนด

ผลลัพธ์จาก Runner จะถูกส่งกลับผ่าน Sentinel-based Result Channel เพื่อช่วยแยกข้อมูลผลการตัดสินออกจาก Output ปกติของโปรแกรมผู้เล่น

บนระบบปฏิบัติการ Windows การจำกัดหน่วยความจำของ SubprocessBackend ทำได้ในลักษณะ Best-effort เนื่องจากไม่มีความสามารถเทียบเท่า setrlimit หรือ Linux cgroup โดยตรง ดังนั้นระดับการควบคุมทรัพยากรจึงอาจไม่เข้มงวดเท่ากับ DockerBackend

9.2 DockerBackend

DockerBackend ใช้ Container Image python:3.14-slim เป็นสภาพแวดล้อมสำหรับประมวลผล Submission และกำหนดข้อจำกัดด้านความปลอดภัยหลายระดับ ได้แก่

- ปิดการใช้งานเครือข่ายด้วย `--network none`
- กำหนด Root Filesystem เป็นแบบ Read-only
- จำกัดการใช้หน่วยความจำผ่าน Memory Cgroup
- จำกัดจำนวน Process ด้วย `--pids-limit 8`
- ยกเลิก Linux Capabilities ทั้งหมดด้วย `--cap-drop ALL`
- เปิดใช้งาน `no-new-privileges`

- รันโปรแกรมด้วยบัญชีผู้ใช้ที่ไม่มีสิทธิ์พิเศษ (Unprivileged User)
- ใช้ /tmp แบบ tmpfs เพื่อหลีกเลี่ยงการเขียนข้อมูลลง Root Filesystem โดยตรง

แนวทางดังกล่าวทำให้ DockerBackend มีระดับการแยกสภาพแวดล้อมและความคุ้มครองทรัพยากรที่เข้มงวดกว่า

SubprocessBackend

9.3 Backend Selection และ Fallback

เมื่อกำหนดให้ระบบใช้ DockerBackend แต่ไม่สามารถเชื่อมต่อกับ Docker Daemon ได้ ระบบจะทำการ Fallback ไปใช้ SubprocessBackend โดยอัตโนมัติ

อย่างไรก็ตาม ผลการตัดสินใจจะต้องรายงาน Backend ตามที่ถูกต้องใช้งานจริง เช่น backend=subprocess เพื่อให้ข้อมูลที่แสดงสอดคล้องกับสภาพแวดล้อมที่ใช้ในการประมวลผล และไม่ทำให้ผู้ใช้งานเข้าใจผิดว่าการประมวลผลเกิดขึ้นภายใน Docker

9.4 AST Guard และขอบเขตด้านความปลอดภัย

ระบบมีการใช้ AST Guard เพื่อตรวจสอบและป้องกันรูปแบบโค้ดบางประเภทก่อนการประมวลผล อย่างไรก็ตาม กลไกดังกล่าวถูกออกแบบให้เป็นเพียง Defence-in-depth หรือชั้นป้องกันเพิ่มเติมเท่านั้น และไม่ถือเป็น Security Boundary หลักของระบบ

ระบบจึงมีตัวเลือก --no-ast-guard สำหรับปิดการทำงานของ AST Guard เพื่อใช้ในการทดลองและแสดงให้เห็นอย่างชัดเจนว่าความปลอดภัยของ Sandbox ไม่ควรขึ้นอยู่กับ การตรวจสอบ AST เพียงอย่างเดียว แนวทางนี้ช่วยให้สามารถประเมินข้อจำกัดของแต่ละมาตรการรักษาความปลอดภัยได้อย่างโปร่งใส โดยไม่ปกปิดความสามารถหรือข้อจำกัดที่แท้จริงของระบบ

10. Application-Layer Protocol Design

กำหนดชื่อโปรโตคอลเป็น CDAP/1.0 (Competitive Development Arena Protocol Version 1.0) โดยใช้รูปแบบ Request-Response ระหว่าง Client และ Arena Server

ตัวอย่างเช่น:

REQUEST

CDAP/1.0 QUEUE

Request-ID: req-001

User: alice

Content-Length: 0

Server ตอบกลับ:

RESPONSE

CDAP/1.0 202 QUEUED

Request-ID: req-001

Content-Type: application/json

```
{"status": "queued"}
```

หรือ Login:

REQUEST

CDAP/1.0 AUTH

Request-ID: req-002

Content-Type: application/json

```
{  
  
  "username": "alice",  
  
  "password": "1234"  
}
```

Response:

RESPONSE

CDAP/1.0 200 OK

Request-ID: req-002

Content-Type: application/json

{

 "authenticated": true

}

11. ตารางสรุป

Method	ทิศทาง	หน้าที่	Response ที่เป็นไปได้
HELLO	Client → Server	เริ่มต้นการเชื่อมต่อและตรวจสอบ Version	200 OK, 426 VERSION_UNSUPPORTED
AUTH	Client → Server	ยืนยันตัวตน	200 OK, 401 AUTH_FAILED
QUEUE	Client → Server	เข้าคิว Matchmaking	202 QUEUED, 409 ALREADY_QUEUED
DEQUEUE	Client → Server	ออกจากคิว	200 OK, 409 NOT_QUEUED
ROOM_CREATE	Client → Server	สร้างห้องส่วนตัว	201 CREATED
ROOM_JOIN	Client → Server	เข้าร่วมห้อง	200 OK, 403, 409 ROOM_FULL
READY	Client → Server	แจ้งสถานะพร้อมแข่งขัน	200 OK, 403 WRONG_STATE

SUBMIT	Client → Server	ส่ง Source Code	202 ACCEPTED, 413, 415
STATUS	Client → Server	ตรวจสอบ Submission	200 OK
VERDICT	Server → Client	แจ้งผล Judge	600-612